

COMP 3659 – Operating Systems

Programming Project 1: A Linux Shell

Overview

In groups of 3, students will develop their own shell program (`mysh`) that provides a command line interface for a Linux user. The `mysh` program is to provide a *subset* of the functionality provided by common Linux shells such as `bash`.¹

Self-enrolment in project groups on D2L:	Monday, September 14 (by 6:00 p.m.)
Team contract and development history (bonus deadline):	Thursday, September 17, end of day
Due date for submission of project deliverables:	Thursday, October 8 (by 11:59 p.m.)
Required group presentation to instructor:	October 19 – 30

By the submission deadline indicated above, groups must submit the source code for a buildable, working shell, together with all supporting files.

Groups are also required to give a presentation to the instructor. The presentation is to include a student-led demonstration of shell functionality (including discussion of limitations and known bugs), discussion of design and implementation issues and decisions, exploration of course concepts that relate to the project, and explanation of how particular details of the `mysh` design, implementation, and functionality relate to these concepts. All participants must be prepared to answer questions from the instructor and to engage in discussion about the project. Successful presentations must demonstrate comprehensive understanding of related course concepts. Presentations require the participation in oral presentation by all group members and require supporting material (e.g., a small number of slides). More information about the presentation requirement will be posted to the course D2L site in a separate document, and groups will select their presentation time slot during the week of October 5-9. Presentations are expected to last approximately 20 (max. 30) minutes and will normally be conducted in person.

Solutions are to make direct use of the necessary Linux system calls, with minimal or no use of additional library functions (see below).

Start Early

Do not wait for groups to be finalized before you start. Read the project description and the marking rubric carefully during week 2, and work out what the project is about, what you already know how to do, and what you will need to learn. A group whose members each arrive having read the requirements and formed an opinion is working by the end of its first meeting. A group that meets in order to find out what the project is loses a week.

¹ See the manual entry for `bash` (`man bash`).

Groups

Students form their own groups of 3 using the group self-enrolment tool on the course D2L site, which opens when this document is released. Self-enrol in a group by 6:00 p.m. on Monday, September 14. Students who have not enrolled by that time will be assigned to a group that has room, and the final groups will be confirmed on Tuesday, September 15. Depending on class size after the add/drop date, there may be one or two groups of 2.

The instructor reserves the right to adjust group membership in order to form appropriately-sized groups, or where a group dynamic has become dysfunctional. In the latter case, members may be required to complete the project individually.

Once groups are formed, each group must agree on a team contract setting out mutual expectations, how the group will meet and communicate, how work will be divided, and how conflicts will be assessed and resolved. A template and a submission link are posted on D2L. Groups must also grant access to their Git repository or the shared development history document (see the Development history paragraph under Development and Assessment).

Bonus: Groups that, by the end of Thursday, September 17, have both submitted a completed contract and given the instructor access to their development history record receive a bonus of 0.15 percent toward the final course grade. Both are required of every group in any case, and a group that completes them later simply forgoes the bonus. The date is deliberately tight, because a contract is only useful if it is agreed before the work starts, and a development history is only useful if it is kept from the beginning.

Normally, the groups formed for project 1 will also be the groups for project 2. Groups should discuss any difficulties as soon as they arise and should approach the instructor if concerns persist.

Functionality Requirements

Basic Functionality

The shell is to run in interactive mode: it must repeatedly prompt the user, read a command line from the standard input, and then cause the command(s) read to be executed (or the shell must respond with an appropriate user error message when execution is not possible). This process is to continue indefinitely, until the user exits the shell (at which point the shell process is to terminate). A sample interaction is shown below:

```
mysh$ /bin/ls
foo.c stuff sometextfile
mysh$ /bin/cat sometextfile
This is the content of sometextfile.
mysh$ exit
```

At each command line, it should be possible for the user to run valid external² commands (or limited sequences of such commands—see below), including commands that may accept zero or more command line arguments. It is also necessary for the shell to support at least one built-in command (`exit`).³

It must be possible for `mysh` to run as an interactive login shell.⁴ In fact, even though `mysh` should be testable by invoking it directly from within a `bash` session, it should also be possible to set up a test user account that uses `mysh` as its default login shell.⁵

Command line format: a command line is to consist of a sequence of zero or more whitespace-delimited “words”, all terminated by a newline. For simplicity, it is acceptable for `mysh` to place a reasonable upper bound on command line length (e.g., it may limit command line length to 256 characters).

It is *not* necessary that the shell provide any sophisticated command-line editing functionality (e.g., `emacs`-like editing using GNU `readline`) or command history mechanism.

It is *not* necessary that the shell read any profile information on start-up.⁶

Background Jobs

By default, user commands are to be run in the foreground, and `mysh` is to suspend execution until command execution terminates. Users must also be able to run commands in the background (i.e., `mysh` will not wait for commands to terminate before proceeding) by ending the command line with an ampersand (&) symbol. The `mysh` functionality and behaviour must be similar to that of `bash` in this regard.

Note: it is relatively easy to implement a basic (and not entirely correct) version of this functionality. This allows the user to create pseudo-background processes, such that the shell process is not properly notified of background job status changes and the user is not able to monitor or control status. Partial functionality will be worth some credit; more complete functionality will be worth additional marks.

Basic I/O Redirection

By default, user commands are to read their input from the standard input and write their output to the standard output. Users must also be able to perform basic input and/or output redirection using less-than and greater-than symbols (<, >). It is not necessary for `mysh` to mimic the full range of `bash` I/O redirection functionality in order to receive an A-range grade, but a reasonable subset of redirection functionality must be supported.

Basic Command Pipelines

Users are to be able to connect the output of one command process to the input of the next by employing the pipe (|) symbol. At a minimum, `mysh` must support two-stage pipelines.

² Here, an *external* command refers to a command that is not built into the shell, but rather is a separate executable file, such as an application or system program.

³ Most commands that are runnable will correspond to commands external to the shell—for example programs like `/bin/ls` and `/bin/cat`.

⁴ See the “invocation” section of the `bash` manual entry for a description.

⁵ For more information, start by reading the manual entry for the `chsh` command.

⁶ E.g., it is not necessary that it provides customization according to `.profile` or `.bashrc`-like files.

Other

Paths: Users should be able to specify any external command using its absolute pathname. Ideally, `mysh` will also support a default search path for commands.

Note carefully that a modern Linux shell is a very sophisticated program that provides a large, diverse set of powerful features. It is *not* necessary for `mysh` to support command completion, lists, compound commands, expressions, coprocesses, shell functions, comments, quoting, parameters (e.g., shell variables), expansion, aliases, evaluation (e.g., arithmetic or conditional), sophisticated job control, shell scripts, or other specialized features of a shell like `bash`.

However, this project is open-ended in that, once the core functionality has been implemented successfully, groups are welcome to continue refining and extending shell capabilities and features as time permits.

Groups that have implemented basic shell functionality as well as support for basic background execution, basic I/O redirection, command pipelines (including of length >2), and command search paths are encouraged to add support for “signals” and to attempt a more sophisticated version of background job handling. Such groups may also or alternatively wish to implement simplified versions of one or two other advanced features (e.g., simple forms of lists).

Implementation Requirements

General

The `mysh` program is to be implemented in C, must be buildable by running “make”, and must be runnable on the Linux VM environment for this course.

Restrictions on the Use of Library Functions

The `mysh` program may not be linked to any libraries other than the C standard library. Further, the final version submitted must endeavour to make minimal or, ideally, *no* use of C standard library functions other than C wrapper functions to Linux system calls.⁷ The intention is that the program relies only on system calls and not on any third-party library functions.

Code and Documentation Standards

Because this is a third year course for computer science majors, groups are expected to understand and follow good design, coding, documenting, and testing practices that yield clear, correct, and readable code that is easy to maintain, modify, extend, and reuse. A code and documentation standard will not be released for this course. Instead, each project group is expected to develop its own code/documentation standards policy document, to include it when submitting the solution, and to briefly describe it during the group presentation.

Note carefully that well-written solutions will be rewarded and poorly written solutions will be penalized.

⁷ This means that the shell is not to rely on dynamic memory allocation implemented via library functions like `malloc` and `free`.

Development and Assessment

Groups are encouraged to start early and to develop their shell incrementally according to the feature order described in the *Functionality Requirements* section. Various lab exercises are designed to provide just-in-time experience with key concepts and system features necessary to complete the project. The instructor may provide additional development guidance as appropriate, including during class time and via material posted to the D2L site.

An overall project mark (also covering assessment of the presentation portion) will be assigned after the group presentations have concluded. Work will be graded in accordance with the grading scheme described in the course outline. Although project work is completed in groups, individual members may receive different marks where contribution or demonstrated understanding differs, as assessed in part through each member's participation in the required presentation.

The functionality portion of the instructor's detailed marking scheme will conform to the general guidelines below:

- Project that correctly implements basic functionality requirements: high F (~45%) to D range
- Project that also correctly implements *basic* background jobs: D+ to C- range
- Project that then also correctly implements basic I/O redirection: C range
- Project that then also correctly implements basic (2-stage) command pipelines: C+ to B- range
- Project that then also correctly implements *n*-stage command pipelines: B range
- Project that then also correctly implements command search paths: B+ range
- Project that then also implements some additional enhancements, up to and including some basic signal handling: A- range
- Project that then also implements more complete and correct signal, terminal, and/or background job handling: A to A+ range
- Project that then implements further shell features: high A+ range (~100%)

However, there will also be major portions of the marking scheme that cover the following:

- Quality of design and implementation
- Evidence of systematic and robust quality assurance practices
- Quality of supporting material (to be submitted as part of the project deliverables), including:
 - Documents related to quality assurance, including a comprehensive test plan
 - Documents related to project governance and management (e.g., the group's team contract; the group's development history record; policies developed by the group, such as design methodology; high-level design documents; code/documentation standards; development timeline; plan for division of labour; evidence of regular group meetings, development activity, and peer review activity, e.g., via minutes or project logs; etc.)
- Quality of accompanying presentation

Development history: Every group must maintain a record of its development work showing who contributed what, and when. Groups using Git must grant the instructor access to the repository at the time it is created; the instructor's GitHub username is posted on D2L. Groups not using Git are responsible for keeping an equivalent record in a Google document shared with the instructor, in which contributions are recorded on the day the work is done. This record forms part of the project

governance material discussed at the presentation, and it is what the instructor will refer to if questions arise about individual contribution. Access must be granted, or the shared document created, before your group starts working on the project.

Self-assessment: Each group may be required to perform self-assessments at one or more points during the project and to share these with the instructor, and groups may be asked to participate in informal self-assessment activities of each other's work.

Miscellaneous

Groups are encouraged to employ software development practices and systems (of their choice) for efficiently and effectively managing their source code and related data. Groups should discuss and make these decisions, and they should document them as part of the project's supporting material. This may include the use of version control software that facilitates collaboration,⁸ as well as processes for safeguarding work through robust backup and recovery protocols.

This document does not provide a full account of all background information needed to complete the project. Groups are expected to start early, manage their time wisely over the time allotted, research relevant Linux and system call details, and seek guidance from the instructor as appropriate.

Submission Instructions

Each group must submit their project source code tree to the instructor by the due date indicated above. All the additional documents must also be submitted by the same due date. Groups seeking an extension should contact the instructor at least one week before the due date. As stated in the course outline, any request must be made before the due date, and additional time granted will not normally exceed 48 hours.

Remember: The `mysh` program must be implemented in C, must be buildable by running “`make`”, and must be runnable on the Linux VM environment for this course.

Once ready to submit, *one* member of the group must do the following:

1. From your personal Linux VM, ensure that your copy of the source code tree is up to date⁹ and ready for submission;
 - a. Perform a final, full rebuild of the project;
 - b. Perform a final test of shell functionality;
2. Delete all auto-generated files, such as “.o” files, the final executable, and auto-backup files generated by text editors;¹⁰
3. Change to a directory one level higher than your project working directory, and then create a compressed archive of that directory using the following commands:

```
cd ..  
tar cf mysh.tar projdir    ← where projdir is your directory
```

⁸ E.g., Git. A guide to using Git in COMP 3659 has been posted to the course D2L site.

⁹ E.g., if using a tool like Git, all contributors should have pushed their changes to the shared repository and then the group member who is submitting should have pulled all such changes.

¹⁰ Ideally, your Makefile supports a “`make clean`” option.

```
gzip mysh.tar
```

You should now have a file named `mysh.tar.gz` in the current working directory.

4. **Using D2L**, upload this file as part of your group's submission for the project.¹¹
 - a. To submit this file, it is easiest to first copy it to your local personal computer (e.g., using a file copy utility like WinSCP) and then to upload it to D2L from there.
5. Also using D2L, upload any additional documents that your group created and maintained during the development process, such as:
 - a. Documents related to quality assurance, including a comprehensive test plan;
 - b. Documents related to project governance and management (e.g., policies developed by the group, such as code/documentation standards; development timeline; plan for division of labour; evidence of regular group meetings and development activity, e.g., via minutes or project logs; high-level design documents; etc.)
 - c. A link to your group's development history record (Git repository or shared Google document), with confirmation that the instructor still has access;
6. Note carefully: it is *not* required to submit supporting material for your presentation (e.g., slides) at this time. Your group will finalize and share these later, in the lead-up to your presentation time slot.

If you have any questions about the submission process, please consult with your instructor. Please also familiarize yourself with the group presentation requirements, which will be posted separately on the course D2L site.

¹¹ Please do not use email to submit your work unless D2L submission is not working.